

Projektpraktikum Regelungstechnik 1: Positionsregelung eines Wagens auf einer schiefen Ebene

mit Low-Cost-Komponenten

Das Praktikum besteht aus mehreren Versuchsteilen, die aufeinander aufbauen. Verschaffen Sie sich bereits vor dem ersten Praktikumstermin einen Überblick über das gesamte Praktikum. Folgen Sie Schritt für Schritt der Anleitung und achten Sie darauf, nichts zu überspringen.

Das vollständige Praktikum ist Prüfungsstoff!

Ziel des Versuches ist es, die Position eines kleinen Wagens auf einer verstellbaren schiefen Ebene zu regeln. Abbildung 1 zeigt den Versuchsaufbau. Die Position des Wagens entlang der Führung wird von einem Ultraschallsensor gemessen. Die Neigung der schiefen Ebene kann durch einen Servo verändert werden, der über eine integrierte Positionsregelung verfügt und direkt über Positionssignale angesteuert wird. Der Sollwert der Wagenposition kann entweder über die Software vorgegeben werden oder über ein lineares Potentiometer.

Der Aufbau des Versuches basiert zu großen Teilen auf aktuellen Rapid-Prototyping-Technologien, wie 3D-Druck, modularer Elektronik und automatischer Code-Generierung.

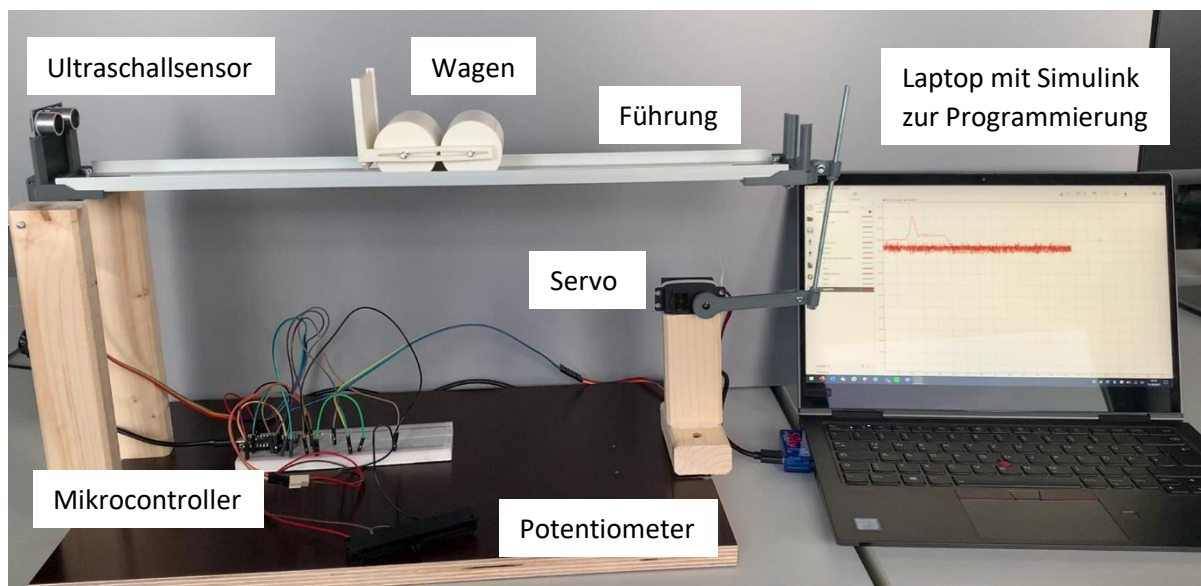


Abbildung 1: Übersicht über den Versuchsaufbau

1. Aufbau des Versuchsstands

Bevor Sie den Versuch durchführen können, bauen Sie sich in Ihrem Team einen Versuchsstand auf. Dabei können Sie sich aussuchen, wie viel Eigenleistung Sie erbringen möchten. Entscheiden Sie sich zuerst für eine der Versionen und lesen Sie dann nur die jeweilige Anleitung. Sie können zu einem späteren Zeitpunkt jederzeit zwischen den Varianten wechseln und fehlende Teile abholen.

In jedem Fall bekommen Sie die folgenden Komponenten von der Hochschule gestellt:

- Mikrocontroller Arduino Nano 33 IoT (geliehen)
- Ultraschall-Sensor HC-SR04 (geliehen)
- Modellbauservo 5521MG mit Positionsregelung (geliehen)
- Linear-Potentiometer (geliehen)
- Steckbrett, Widerstände und Kabel (geliehen)

Beachten Sie, dass der Mikrocontroller ein 3,3V-Controller ist – d. h., dass an den digitalen und analogen Eingängen maximal 3,3V anliegen dürfen! Zu hohe Pegel zerstören den Controller!

Da der Ultraschall-Sensor mit 5V gespeist werden muss, muss der Pegel dessen Echo-Signals mit einem Spannungsteiler von 5V auf 3,3V abgesenkt werden, bevor das Signal im Mikrocontroller ausgewertet wird.

Abbildung 2 zeigt das Pinout-Schema des verwendeten Arduino-Moduls. Abbildung 3 zeigt den schematischen Aufbau des Versuchsstandes.

Hinweis: Sollten Sie ein eigenes Arduino-Modul verwenden, müssen Sie wahrscheinlich eine Brücke auf das Modul löten, um den 5V-Ausgang zu aktivieren.

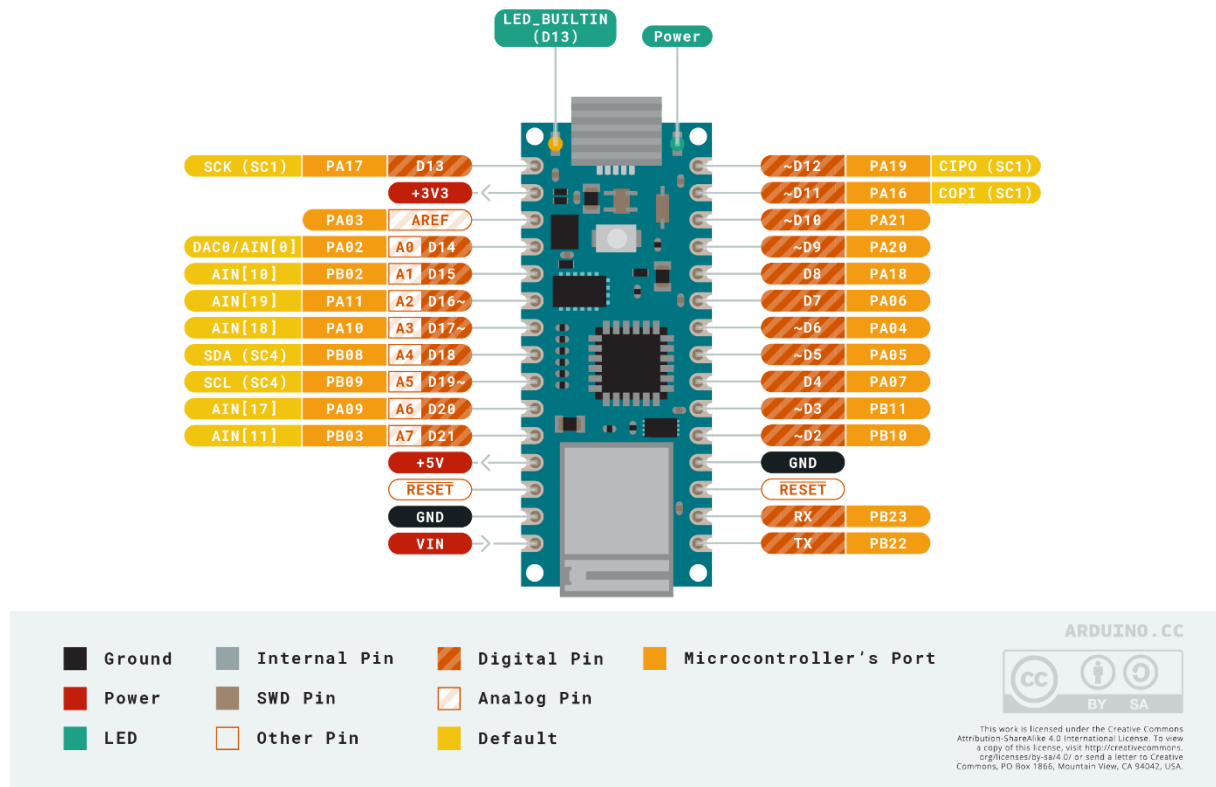


Abbildung 2: Pinout-Schema des verwendeten Arduino-Moduls

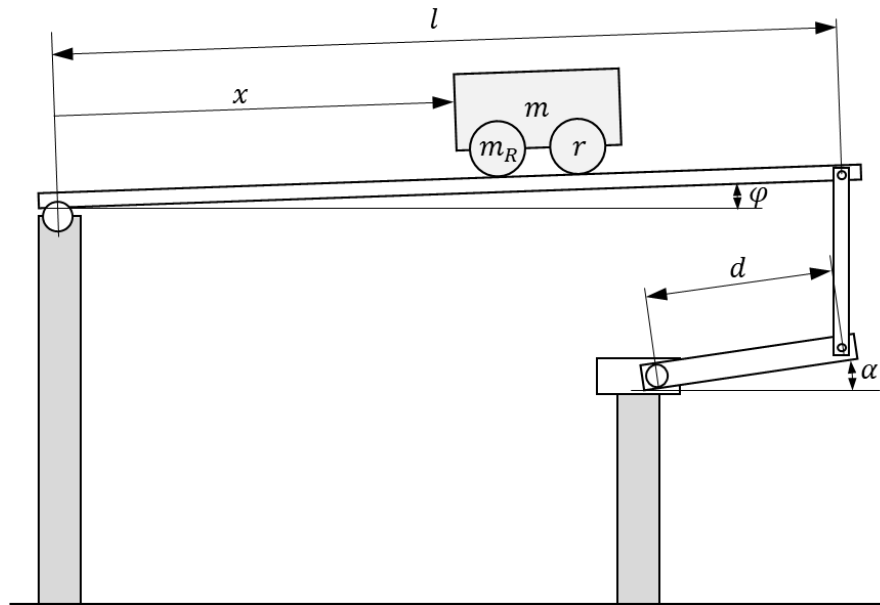


Abbildung 3: Schematische Darstellung des Aufbaus

Verbinden des Arduino mit Simulink

Zur Programmierung des Mikrocontrollers verwenden wir die automatische Code-Generierung aus Simulink. Gehen Sie zum Herstellen der Verbindung zwischen dem Arduino und Simulink (auf Windows 10) wie folgt vor:

1. Deinstallieren Sie ggfs. alte Versionen der Arduino-Umgebung auf Ihrem Rechner.
2. Installieren Sie die aktuelle Version der Arduino-Umgebung und starten Sie diese. Verbinden Sie den Arduino mit Ihrem Rechner und es erscheint ein Hinweis, dass das „Arduino SAMD Boards“ Paket zur Nutzung Ihres Arduino benötigt wird. Installieren Sie dieses. Ihr Arduino sollte im Gerätemanager von Windows nun als „Arduino NANO 33 IoT“ unter „Anschlüsse“ gelistet werden.
3. Installieren Sie MATLAB und Simulink mit (mindestens) folgenden Paketen (falls nicht schon vorhanden):
MATLAB; Simulink; Control System Toolbox; Embedded Coder; MATLAB Coder; Simulink Coder; Symbolic Math Toolbox; System Identification Toolbox.
4. Öffnen Sie dann MATLAB und dort den Add-On Explorer aus der oberen Symbolleiste. Suchen Sie das Paket „Simulink Support Package for Arduino Hardware“ und installieren Sie es. Klicken Sie am Ende der Installation auf „Setup later“.
5. Erzeugen Sie ein neues Simulinkmodell mit folgenden Einstellungen in den Model Settings:
Stop time: inf; Solver Type: Fixed Step; Solver: discrete (no continuous states); Fixed-step size: 0.005. Wählen Sie unter „Hardware Implementation“ als „Hardware Board“ den „Arduino Nano 33 IoT“.
6. Erstellen Sie für einen ersten Test ein Modell, das den Ultraschallsensor ausliest und eine feste Position an den Servo kommandiert. Die notwendigen Simulink-Blöcke „Standard Servo Write“ und „Ultrasonic Sensor“ finden Sie im „Library Browser“ im Abschnitt „Simulink Support Package for Arduino Hardware“. Konfigurieren Sie alle Blöcke und führen Sie Ihr Modell anschließend durch einen Klick auf „Monitor & Tune“ im Register „Hardware“ aus.
7. Sollten Sie Probleme mit der Verbindung haben, so testen Sie ein anderes USB-Kabel und prüfen Sie nochmals, ob Schritt 2 erfolgreich abgeschlossen wurde.

Variante 1: Eigener Entwurf (größter Aufwand)

Entwerfen Sie den Versuchsstand nach Ihren eigenen Vorstellungen und setzen Sie Ihr eigenes Konzept um. Nutzen Sie Holz, 3D-Druck-Teile und sonstige Materialien und erstellen Sie einen sinnvollen Gesamtaufbau. Beachten Sie, dass der mögliche Fahrweg des Wagens mindestens 40cm betragen sollte und dass der Wagen an seiner Vorderseite eine gerade und glatte Fläche von ca. 5x5cm benötigt, um vom Ultraschall-Sensor zuverlässig erkannt zu werden.

Behalten Sie auch die grundsätzliche Struktur des Aufbaus wie in Abbildung 3 gezeigt auf jeden Fall bei. (Sollten Sie keinen Zugriff auf einen 3D-Drucker haben, können Sie die Drucker der Hochschule verwenden. Erstellen Sie dazu **druckbare** STL-Dateien und setzen Sie sich mit Ihrem Praktikumsbetreuer in Verbindung.)

Beschaffen Sie sich alle nötigen Datenblätter im Internet und bauen Sie die erforderliche Schaltung rund um den zur Verfügung gestellten Arduino auf dem Steckbrett auf.

Variante 2: Eigene Konstruktion nach Vorgaben (mittlerer Aufwand)

Sie erhalten neben den o.g. Bauteilen folgende Komponenten:

- Die Grundplatte und die Ständer aus Holz
- Die Aluminiumschienen zum Aufbau der Führung

Die notwendigen 3D-Druck-Teile für den Aufbau, sowie den Wagen konstruieren und drucken Sie selbst. (Sollten Sie keinen Zugriff auf einen 3D-Drucker haben, können Sie die Drucker der Hochschule verwenden. Erstellen Sie dazu druckbare STL-Dateien und setzen Sie sich mit Ihrem Praktikumsbetreuer in Verbindung.)

Beachten Sie,

- dass der mögliche Fahrweg des Wagens mindestens 40cm betragen sollte
- dass der Wagen an seiner Vorderseite eine gerade und glatte Fläche von ca. 5x5cm benötigt, um vom Ultraschall-Sensor zuverlässig erkannt zu werden
- dass der Hebelarm am Servo ca. 10cm betragen sollte
- dass Sie die vertikale Verbindung zwischen dem Hebel am Servo und der Führung einstellbar ausführen sollten, um den Nullpunkt justieren zu können

Setzen Sie die in Abbildung 4 gezeigte Schaltung mit den gegebenen Komponenten um.

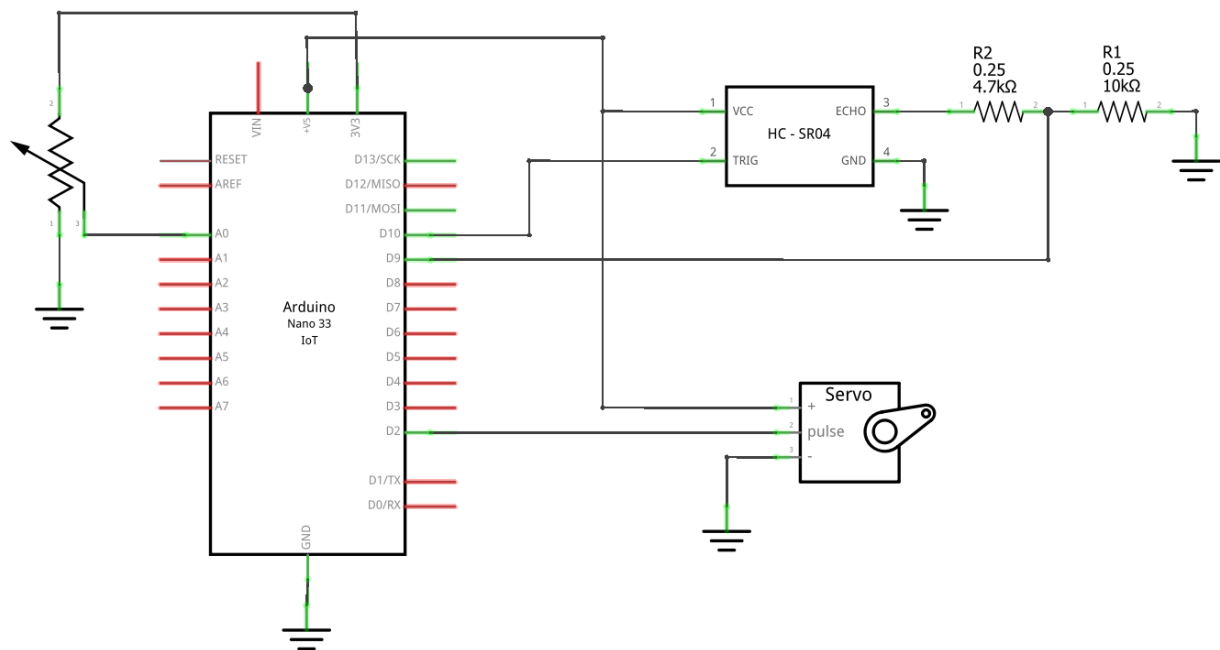


Abbildung 4: Schaltplan der notwendigen Elektronik

Achtung: Befestigen Sie den Hebel am Servo erst nachdem die Software lauffähig ist und Sie eine Verbindung zum Rechner hergestellt haben. Sie müssen bei der Montage des Hebels sicherstellen, dass dieser in etwa waagrecht steht, wenn der Servo auf 90° steht. Wie Sie die Verbindung zwischen dem Aufbau und Simulink herstellen, erfahren Sie im Abschnitt Simulation → Modellidentifikation weiter unten.

Variante 3: Zusammenbau der gegebenen Komponenten (geringster Aufwand)

Sie erhalten alle für den Versuchsstand nötigen Bauteile und setzen diese zusammen. Die folgenden Bilder zeigen einige Details des Zusammenbaus.



Abbildung 5: Details des Versuchsaufbaus

Achtung: Befestigen Sie den Hebel am Servo erst nachdem die Software lauffähig ist und Sie eine Verbindung zum Rechner hergestellt haben. Sie müssen bei der Montage des Hebels sicherstellen, dass dieser in etwa waagrecht steht, wenn der Servo auf 90° steht.

Bauen Sie die Schaltung entsprechend Abbildung 6 und Abbildung 7 aus den gegebenen Komponenten auf.

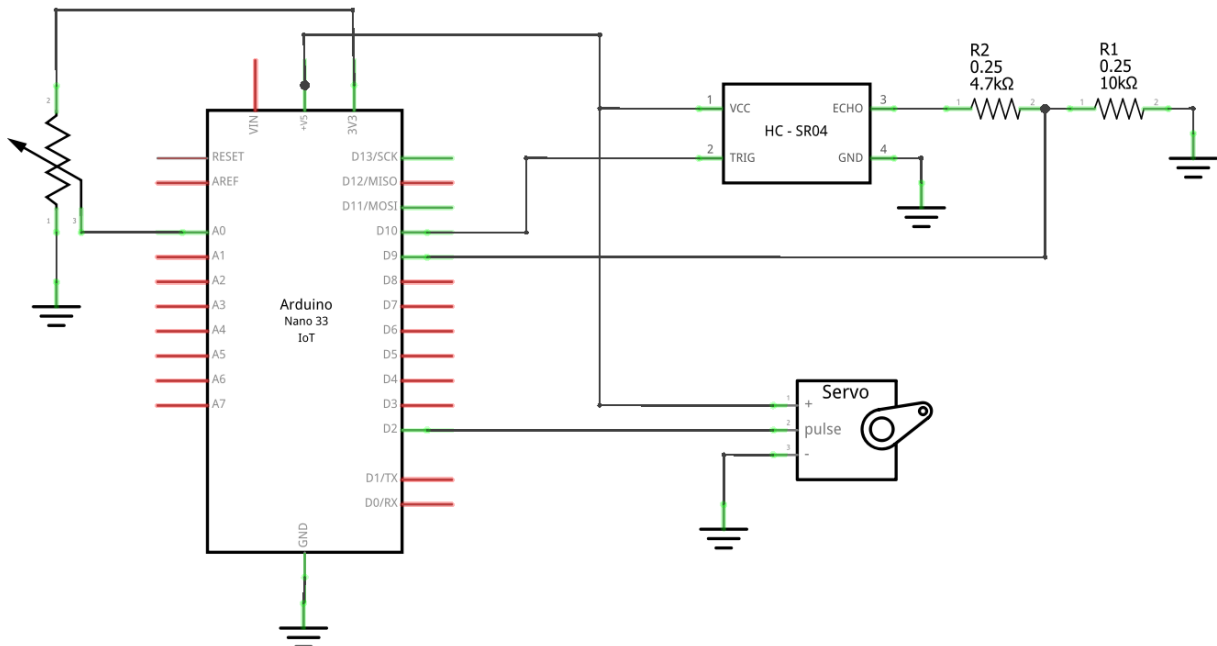


Abbildung 6: Schaltplan der notwendigen Elektronik

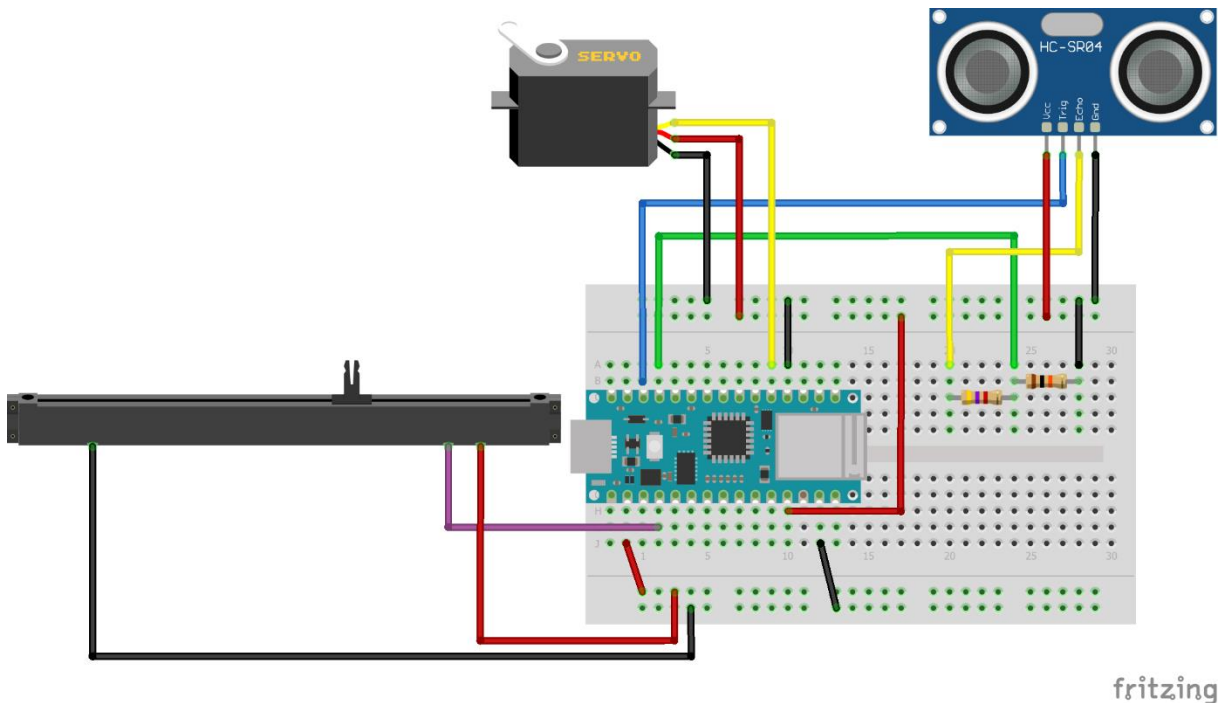


Abbildung 7: Aufbau der Elektronik auf einem Steckbrett

2. Modellierung

Nun soll ein dynamisches Modell des Versuchsaufbaus erstellt werden, mit dem sowohl das Verhalten des Systems als auch das der Regelung in der Simulation untersucht werden kann. Nutzen Sie die schematische Darstellung des Versuchstands und die Bezeichnungen aus Abbildung 3 um ein allgemeines dynamisches Modell (zunächst ohne Zahlenwerte) herzuleiten. Es gelten folgende Annahmen:

- Die Masse des Wagens ohne die beiden Rollen/Räder ist m
- Die Rollen des Wagens sind ideale Zylindermantel (d.h. die gesamte Masse konzentriert sich an der Oberfläche) der Masse m_R , mit Radius r
- Der Wagen rollt ohne Reibungsverluste und ohne zu gleiten
- Der senkrechte Abstand zwischen den beiden Drehpunkten des senkrechten Hebels bleibt immer gleich (d.h. die Drehung des Hebels kann vernachlässigt werden).
- Die Zentripetalkraft auf den Wagen, die sich theoretisch aus der Drehgeschwindigkeit der schiefen Ebene ergibt kann vernachlässigt werden ($\dot{\varphi} \ll 1$, quasi-stationäre Winkelposition der Ebene).
- Die Dynamik des Servos kann vernachlässigt werden. Wir gehen also davon aus, dass eine Vorgabe für den Winkel α sofort umgesetzt wird. Der Winkel α ist damit direkt die Eingangsgröße in das System.

Berücksichtigen Sie im Modell sowohl die Rotationsträgheit der Räder als auch die Massenträgheit des Wagens und nutzen Sie die entsprechenden Kräfte- und Momentengleichgewichte. Das Eingangssignal des Systems ist die Winkelstellung α des Servos, der interessierende Ausgang des Systems ist die Wagenposition x .

? Handelt es sich bei dem sich ergebenden Modell um ein lineares oder ein nichtlineares Modell?

Nutzen Sie nun die Kleinwinkelnäherung $\sin \alpha \approx \alpha$ und $\cos \alpha \approx 1$ für alle in der Modellgleichung verbleibenden Winkel.

? Ist das nun resultierende System linear oder nichtlinear?

? Welche Ordnung hat das System?

3. Simulation

Nutzen Sie nun Simulink zur Simulation des Verhaltens des vorliegenden Systems. Untersuchen Sie dabei auch die Unterschiede zwischen linearem und nichtlinearem Modell. Achten Sie darauf, alle Signale und Systeme in Simulink ordentlich zu bezeichnen.

Setzen Sie die folgenden Werte für die Konstanten ein:

$m = 0,023 \text{ kg}$	$m_R = 0,027 \text{ kg}$	$r = 0,021 \text{ m}$
$l = 0,5 \text{ m}$	$d = 0,1 \text{ m}$	

Lineares Simulationsmodell

Erstellen Sie ein neues Simulinkmodell mit folgenden Einstellungen:

- Stop time: 2s
- Solver Type: Variable Step, ode45 (Dormand-Prince)¹
- Solver → Solver Details → Relative Tolerance: 1e-6

Bauen Sie Ihr Modell in SI-Einheiten auf und berücksichtigen Sie, dass Simulink intern in RAD rechnet, Sie die Servostellung aber in DEG vorgeben müssen. Weiterhin kann der Servo einen Winkelbereich von 0-180° anfahren. Nachdem sowohl positive wie auch negative Winkelausschläge benötigt werden, muss die Nullposition für α auf eine Servostellung von 90° gesetzt werden. Berücksichtigen Sie auch diesen Offset bereits in Ihrem Modell und bauen Sie es so auf, dass das Eingangssignal die Servoposition ist.

Initialisieren Sie alle für die Modellierung notwendigen Konstanten in einem MATLAB-File und binden Sie dieses als „InitFcn“ (Modeling → Model Explorer → Callbacks → InitFcn) in Ihre Simulation ein. Damit bewirken Sie, dass vor jeder Simulation die Werte entsprechend Ihrem Skript gesetzt und dann für die Simulation verwendet werden. So können Sie alle Konstanten an einer zentralen Stelle anpassen.

Setzen Sie den Anfangswert für die Wagengeschwindigkeit auf $\dot{x}(0) = 0$ und den für die Position auf $x(0) = 0,25 \text{ m}$.

Nutzen Sie als Eingangssignal in Ihre Simulation einen Sprung, der bei 90° startet (also der horizontalen Stellung) und nach einer Sekunde auf den über Ihr Skript definierten Endwert SERVO_CMD springt. Führen Sie die Simulation dann für Werte für SERVO_CMD von 92° bis 100° in 2°-Schritten und anschließend von 100° bis 150° in 10°-Schritten aus.

? Wie verhält sich der Wagen? Entspricht das Ihren Erwartungen?

? Bei welchem Stellwinkel erreicht der Wagen genau nach einer Sekunde das Ende der Führung?

¹ Über die Einstellung „Variable Step“ aktivieren Sie Simulationsalgorithmen, die selbstständig entscheiden, mit welcher Schrittweite die Simulation durchgeführt wird. Diese sog. **Schrittweitensteuerung** basiert auf zwei verwandten Runge-Kutta-Simulations-Algorithmen mit unterschiedlicher Ordnung. Während der Simulation wird die Schrittweite so lange verkleinert, bis der Fehler, der sich durch Vergleich der Ergebnisse der beiden Simulationsmethoden ergibt, den eingestellten Grenzwert unterschreitet. Nur dann wird der berechnete Simulationsschritt verwendet. Andernfalls wird er verworfen und es wird mit einer kleineren Schrittweite nochmals gerechnet. Werden mehrere Schritte in Folge sofort akzeptiert, wird in darauffolgenden Schritten nach und nach die Schrittweite wieder vergrößert (solange die Fehlerbedingung erfüllt bleibt). Die Schrittweitensteuerung rechnet damit auch in Zeitabschnitten der Simulation, in denen „sich viel tut“ genau, ohne in Abschnitten „mit geringer Veränderung“ unnötig viele Berechnungen durchzuführen. Insgesamt wird so eine Verkürzung der Rechenzeit der Simulation erreicht, ohne dass es zur Verschlechterung in der Genauigkeit kommt.

Nichtlineares Simulationsmodell

Verwenden Sie die Simulation aus dem vorherigen Abschnitt weiter und verpacken Sie Ihre lineare Simulation in ein Subsystem (die betreffenden Blöcke auswählen und das Rechtsklick-Menü verwenden).

Kopieren Sie anschließend dieses Subsystem und erweitern Sie es so, dass es das ursprüngliche, nichtlineare Modell simuliert.

Nutzen Sie für beide Subsysteme in Ihrer Simulation nun dasselbe Eingangssignal und vergleichen Sie die jeweiligen Ausgangssignale auf zwei verschiedene Arten:

- Fügen Sie einerseits einen MUX-Block ein und nutzen Sie ein Scope, um die so zusammengeführten Signale auf einer gemeinsamen Zeitachse betrachten zu können.
- Aktivieren Sie für die beiden Signale die Funktion „Log Selected Signals“ und vergleichen Sie die Signalverläufe im Signal Inspector von Simulink.

? Für welche Servo-Endstellungen stellen Sie relevante Abweichungen in den Verläufen fest?

? Denken Sie, dass eine Reglerauslegung mit dem linearen Modell funktionieren kann?

Modellidentifikation

Nun ermitteln Sie näherungsweise die Parameter Ihres eigenen Systems.

Bestimmen Sie dazu zunächst die im Modell nötigen Konstanten für Ihren Versuchsaufbau mit Hilfe einer Waage und eines Messschiebers bzw. Meterstabes und ändern Sie die Werte in Ihrem Initialisierungsfile entsprechend.

Erstellen Sie nun ein neues, zweites Simulink-Modell, das Sie zur Arbeit mit dem Versuchsaufbau verwenden. Konfigurieren Sie die Modelleinstellungen wie folgt:

- Stop time: inf (unendlich lange Simulationszeit, die Verbindung zum Versuchsstand kann mit dem Stop-Button getrennt werden)
- Solver Type: Fixed Step
- Solver: discrete (no continuous states)
- Fixed-step size: 0.005 (= 5ms)

Mit der Einstellung „Solver: discrete (no continuous states)“ erreichen Sie, dass Simulink keinerlei Simulation für kontinuierliche Zustände mehr durchführt, also keinen numerischen Integrationsalgorithmus einsetzt. Dies ist notwendig, da der Mikrocontroller sonst nicht leistungsstark genug ist, um den Code in Echtzeit ausführen zu können.

Konfigurieren Sie weiterhin die Kommunikation zwischen Simulink und dem Arduino in den Einstellungen des Simulationsmodells im Abschnitt „Hardware Implementation“. Wählen Sie dort unter „Hardware Board“ den „Arduino Nano 33 IoT“². Alle weiteren Einstellungen können Sie auf den Standardwerten belassen.

Die Simulink-Blöcke zur Kommunikation mit den realen Ein- und Ausgängen des Systems finden Sie im „Library Browser“ von Simulink im Abschnitt „Simulink Support Package for Arduino Hardware“. Der Block zur Ansteuerung des Servos heißt „Standard Servo Write“, der zum Auslesen des

² Sollte Ihnen das Board nicht in der Auswahlliste angezeigt werden, klicken Sie auf „Get Hardware Support Packages“ in der Liste und installieren Sie das Paket „Simulink Support Package for Arduino Hardware“.

Ultraschallsensors heißt „Ultrasonic Sensor“. Bauen Sie ein Modell auf, mit dem Sie eine Vorgabe für die Servoposition machen und die Position des Wagens messen können. Setzen Sie die Ein-/Ausgangspins in den Blöcken und setzen Sie an allen Stellen die Sample Time auf 5ms (In vielen Blöcken in Simulink bewirkt eine Sample Time von -1, dass die Sample Time der übergeordneten Einheit verwendet wird; entspricht „inherited“).

Führen Sie Ihr Modell anschließend durch einen Klick auf „Monitor & Tune“ im Register „Hardware“ aus und untersuchen Sie das Verhalten bei denselben Sprunganregungen wie in der Simulation oben. In diesem Modus können Sie Parameter innerhalb des Simulink-Modells verändern, die direkt an die Hardware übertragen und dort umgesetzt werden.

? Was stellen Sie fest? Deckt sich das Verhalten des realen Systems mit der Simulation?

? Wo vermuten Sie ggfs. Probleme?

Hinweis: Sollten Sie bei der Ausführung Ihres Modells im Zusammenspiel mit der echten Hardware Schwierigkeiten haben, dann erhöhen Sie die Übertragungsgeschwindigkeit zwischen der Hardware und der Simulation mit folgendem Befehl in MATLAB:

```
codertarget.arduino.base.registry.setBaudRate(gcs, 230400)
```

Hinweis: Sollte Ihre Simulation nicht in Echtzeit ausgeführt werden, prüfen Sie nochmals, ob Sie alle Ein-/Ausgangs-Pins in den Blöcken korrekt gesetzt haben.

Hinweis: Möchten Sie die Eingangswerte „manuell“ verändern, empfiehlt es sich statt des „Step“-Blockes einen „Constant“-Block zur Signalerzeugung zu verwenden, da Sie so den Vorgabewert für die Servoposition zur Laufzeit einfach anpassen können.

Hinweis: Hin und wieder kommt es vor, dass Simulink die Verbindung zur Hardware verliert, bzw. dass der PC oder der Mikrocontroller nicht mit der Verarbeitung der Daten nachkommen. Starten Sie in diesem Fall die Simulation einfach erneut. Sollten diese Probleme verstärkt auftreten, stellen Sie einerseits sicher, dass auf Ihrem PC keine Hintergrundaufgaben laufen und verlängern Sie ggfs. die „Step Time“ in der Simulation.

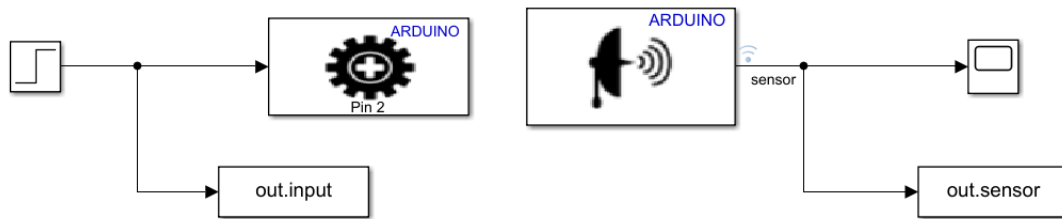
Reibungseffekte

Erweitern Sie Ihr Modell nun so, dass es auch ganz allgemeine, geschwindigkeitsabhängige Reibungseffekte abbilden kann. Den entsprechenden Parameterwert werden wir im nächsten Schritt von Simulink durch eine Parameteridentifikation abschätzen lassen.

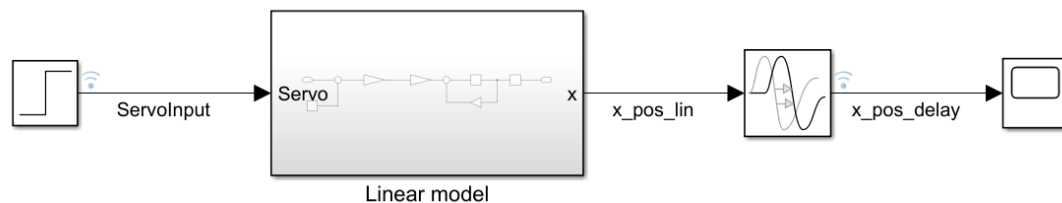
Bei der Parameteridentifikation werden Messdaten aufgezeichnet und anschließend die Parameter der Simulation so lange (systematisch) angepasst, bis das Simulationsergebnis mit den aufgezeichneten Messdaten übereinstimmt. Gehen Sie dazu in Simulink wie folgt vor:

1. Öffnen Sie Ihr Simulink-Modell zur Ansteuerung des Prüfstandes und stellen Sie als Eingangssignal einen Sprung von 90° auf 120° nach 1 Sekunde ein. Setzen Sie die gesamte Simulationszeit auf 3 Sekunden (Stop Time).
2. Achten Sie darauf, dass sich der Wagen am Anfang 25cm vom Sensor entfernt befindet (messen Sie die Position mit dem Sensor und markieren Sie die Stelle an der Führung).
3. Fügen Sie zwei „To Workspace“-Blöcke zu Ihrem Modell hinzu, um die Servo- und Positionsdaten abspeichern zu können. Setzen Sie die Namen der Variablen auf „input“ und „sensor“ und wählen Sie als Save format: „Structure With Time“.

4. Führen Sie dann die Messung durch. Sie finden die aufgezeichneten Daten im MATLAB-Workspace unter den von Ihnen festgelegten Variablennamen.



5. Nutzen Sie den folgenden MATLAB-Code, um die Messdaten in das für die Identifikation notwendige Format zu überführen:
`identdata = [sensor.time, squeeze(sensor.signals.values)];`
 Kürzen Sie das aufgezeichnete Signal dann an der Stelle, an der der Wagen den Endanschlag erreicht hat. Suchen Sie dazu den Zeitpunkt in den Messdaten und ermitteln Sie den entsprechenden Index in „identdata“. Überschreiben Sie „identdata“ mit einer gekürzten Version.
6. Erstellen Sie ein neues Simulink-Modell, das das lineare Modell, erweitert um den Reibungseinfluss, enthält. Stellen Sie sicher, dass derselbe Eingangsverlauf eingestellt ist wie am Prüfstand.
7. Fügen Sie einen Transport-Delay-Block ein. Setzen Sie den „Time delay“ über eine Variable, deren Wert Sie mit 0 initialisieren. Den „Initial output“ setzen Sie auf 0,25 (entsprechend der anfänglichen Wagenposition).
8. Auch die Konstante für den Reibungseinfluss setzen Sie auf eine mit 0 initialisierte Variable. Diese beiden Parameter lassen Sie Simulink im nächsten Schritt ermitteln. Weiterhin aktivieren Sie das Signal Logging für das verzögerte Positionssignal.



9. Öffnen Sie den Reiter „Apps“ und suchen Sie „Parameter Estimator“. Sollten Sie das entsprechende Paket noch nicht installiert haben, klicken Sie im selben Register auf „Get Add-Ons“ und installieren Sie „Simulink Design Optimization“.
10. Klicken Sie im sich öffnenden Fenster auf „Select Parameters“ und wählen Sie den Reibungsparameter und den Parameter, den Sie als „Delay time“ eingestellt haben. Damit weisen Sie Simulink an, diese beiden Parameter zu bestimmen.
11. Klicken Sie auf „New Experiment“ um dem Ausgangsverlauf die Messdaten zuzuordnen. Wählen Sie unter „outputs“ NUR den Ausgang der Simulation und entfernen Sie ggfs. alle anderen Signale. Tippen Sie in das Feld unter dem Ausgang „identdata“ ein. Schließen Sie das Fenster mit „OK“ nach einem Klick auf „Plot & Simulate“.
12. Starten Sie die Berechnung dann mit einem Klick auf „Estimate“. Sie sehen während der Berechnung, wie sich die Simulation immer weiter der Aufzeichnung annähert.
13. Übertragen Sie die Ergebniswerte in das Initialisierungsskript Ihrer Simulation.

Damit haben Sie die Simulation für die Reglerauslegung vorbereitet.

4. Entwicklung der ersten Regelung

Nutzen Sie für die Auslegung der Regelung das lineare Modell mit Reibungseinfluss.

Transformieren Sie die Systemdynamik für den Reglerentwurf in den Laplace-Bereich. Da die Strecke bereits integrierendes Verhalten hat, ist davon auszugehen, dass mit einem PD-Regler gutes Regelverhalten erreicht werden kann. Bestimmen Sie mit der Methode der Polplatzierung die Reglerparameter für eine allgemeine Vorgabe von Kennkreisfrequenz ω_0 und Dämpfung D des Wunschkpolynoms zweiter Ordnung.

Platzieren Sie anschließend die Pole der Regelung so, dass der geschlossene Kreis eine Kennkreisfrequenz von $\omega_0 = 4$ und eine Dämpfung von $D = 1$ hat.

? Welche Verstärkungsfaktoren erhalten Sie für die beiden Regleranteile?

Implementieren Sie Ihren Regler in der linearen Simulation mit Reibungseinfluss. Achten Sie darauf, den Sollwinkel α (der sich aus den theoretischen Betrachtungen ergibt) in der Simulation korrekt in die Winkelvorgabe für den Servo umzurechnen. Wählen Sie als Sollposition einen Sprung von 0,25m auf 0,1m. Simulieren Sie das System einmal mit Totzeit (Transport Delay) und einmal ohne (nutzen Sie die Funktion „Comment Through“, um die Wirkung des Totzeit-Blockes zu deaktivieren).

? Wie wirkt sich die Totzeit auf das Verhalten des Reglers aus?

Ermitteln Sie mit Hilfe der Simulation eine Kennkreisfrequenz und eine Dämpfung, bei der die Regelung auch unter Berücksichtigung der Totzeit funktioniert.

Implementieren Sie diese Regelung nun für den realen Prüfstand. Erweitern Sie dazu das Simulationsmodell, das Sie für die Aufzeichnung der Messdaten verwendet haben. Fügen Sie zur Sicherheit noch einen Saturation-Block mit Limits von 15 und 155 (ermitteln Sie ggfs. die in Ihrem Aufbau maximal möglichen Ausschläge durch Probieren) vor dem Servo-Output-Block ein, um zu verhindern, dass der Servo oder der Hebel am Aufbau anschlagen.

Führen Sie dann das Modell aus, ändern Sie die Sollwertvorgabe und stören Sie die Position des Wagens, in dem Sie diesen mit dem Finger verschieben.

? Was stellen Sie fest? Sind Sie mit dem Verhalten der Regelung zufrieden? Vergleichen Sie auch den Verlauf der Sollwertvorgabe mit dem Verlauf der Wagen-Ist-Position.

? Experimentieren Sie mit verschiedenen Werten für ω_0 und D . Was stellen Sie fest?

Untersuchen Sie anschließend in der Simulation und am realen Versuchsstand, wie der geschlossene Kreis auf eine sinusförmige Sollwertvorgabe unterschiedlicher Frequenzen reagiert.

? Entspricht dies Ihren Erwartungen?

Realisieren Sie im letzten Schritt die Sollwertvorgabe über das Linearpoti. Schauen Sie sich an welche Werte Sie über den Verstellbereich des Potis erhalten und überlegen Sie sich eine geeignete Umrechnung auf den Sollwertbereich. Wenn Sie die Möglichkeit beibehalten möchten auch Sollwerte über Simulink vorzugeben, verwenden Sie einen „Manual Switch“.

? Welches Problem ergibt sich aus der Kombination des analogen Eingangs mit dem D-Anteil der Regelung? Lässt sich das System gut stabilisieren?

5. Erweiterung der Regelung

I-Anteil

Zur Verbesserung der stationären Genauigkeit soll der Regelung nun ein I-Anteil hinzugefügt werden. Experimentieren Sie zuerst mit der Simulation (mit und ohne Totzeit) und ermitteln Sie einen geeigneten Verstärkungsfaktor für den I-Anteil.

Erweitern Sie dann das Modell für die reale Hardware um einen I-Anteil und experimentieren Sie mit dem realen System. Da die numerische Integration im Modell deaktiviert wurde, müssen Sie an dieser Stelle statt des normalen „Integrator“-Blockes einen „Discrete-Time Integrator“ verwenden.

Signalfilterung

Um das Rauschen des Sensors zu reduzieren, kann ein dynamisches Filter (dynamisches System, dessen Verhalten simuliert wird) eingesetzt werden. In diesem Beispiel soll ein Filter zweiter Ordnung (PT2-System) verwendet werden. Dies hat den Vorteil, dass auch die geglättete Ableitung des Sensorsignals direkt aus dem Filter gewonnen werden kann.

Zur Signalfilterung bietet es sich an, die Dämpfung des PT2-Systems zu $D = 1$ zu wählen. Ersetzt man ω_{0f} durch die Zeitkonstante $T_f = \frac{1}{\omega_{0f}}$, so wird die Differentialgleichung des Systems zu:

$$T_f^2 \cdot \ddot{y}_f(t) + 2 \cdot T_f \cdot \dot{y}_f(t) + y_f(t) = y(t)$$

Oder im Frequenzbereich:

$$G_f(s) = \frac{Y_f(s)}{Y(s)} = \frac{1}{(T_f s)^2 + 2 \cdot T_f s + 1}$$

Damit bleibt zur Konfiguration des Filters noch ein Freiheitsgrad übrig, nämlich T_f . Überlegen Sie mit Hilfe Ihres Wissens über die Sprungantwort und den Frequenzgang eines PT2-Systems (mit Dämpfung $D = 1$), wie sich T_f auf das Verhalten des Ausgangssignals des Filters auswirkt.

? In welcher Größenordnung vermuten Sie geeignete Werte zur Filterung des Positions-Sensorsignals?

? Wie würden Sie das Filter zur Verarbeitung des Sollwertes einstellen?

Implementieren Sie beide Filter im realen Versuchsaufbau und experimentieren Sie mit den Zeitkonstanten. Passen Sie weiterhin die Struktur der Regelung so an, dass der D-Anteil nicht mehr durch numerische Differentiation ermittelt wird, sondern die Ableitungen aus den Filtern verwendet.

Programmierung im Stand-Alone Mode

Kombinieren Sie nun alle Ihre Erkenntnisse von oben und versuchen Sie eine möglichst ruhige, stabile Regelung für die Wagenposition umzusetzen.

Wenn Sie mit der Entwicklung Ihrer Regelung fertig sind und keine Aufzeichnung bzw. Ausgabe der Signalverläufe am Rechner mehr benötigen, können Sie den von Ihnen entwickelten Regler über einen Klick auf „Hardware“ → „Build, Deploy & Start“ auf den Arduino flashen und dort im Stand-Alone Modus ausführen. Jetzt benötigt Ihre Regelung keinen Kontakt mehr zu Simulink, um zu arbeiten. Eine Stromversorgung über USB reicht nun aus, um den Prüfstand zu betreiben.

Da die Kommunikation mit dem Rechner relativ viel Rechenleistung des Microcontrollers erfordert, könnten Sie nun auch die Regelungsschrittweite (Step time) noch weiter verkleinern.

Anhang

Stückliste des Versuchsaufbaus

Für Variante 1:

- Elektronik
 - 1 Arduino Nano IoT
 - 1 Steckbrett
 - 1 Linearpoti
 - 1 Kabelsatz
 - 2 Widerstände, 1x 10k, 1x 4,7k
- 1 Servo
- 1 Ultraschallsensor

Für Variante 2 zusätzlich:

- Aufbau
 - 2 Alu-Schienen
 - 1 Grundplatte
 - 2 lange Holzständer
 - 1 kurzer Holzständer
 - 1 Kabelbinder

Für Variante 3 zusätzlich:

- Aufbau
 - 1 Gewindestange M4
 - 2 Endteile für die Alu-Schienen
 - 1 Kunststoff-Drehlager für Alu-Schienen
 - 1 Lange Alu-Achse
 - 1 Servo-Arm
 - 2 unterschiedliche Winkelanschlüsse für die Gewindestange
- Wagen:
 - 2 Rollen
 - 2 kurze Alu-Achsen
 - 2 Seitenteile
 - 1 Frontplatte
- Schrauben
 - 3 Holzschrauben
 - 5 Schrauben M4
 - 9 Muttern M4